

NEUQuest: A Dual-Native Cross-Platform Architecture for Student Event Discovery and AI-Assisted Budget Trip Planning

Kaustubha Venkata Eluri
Northeastern University

Abstract—Northeastern University students discover campus and off-campus events through a fragmented set of tools – Eventbrite for paid events, club Discord servers for free ones, the official NEU events portal, and Facebook groups for last-minute meetups – with no single feed tuned to student budgets or student interests. This paper presents NEUQuest, a student-exclusive mobile application consisting of two independently implemented native clients, a Java/Material 3 Android application and a SwiftUI/MVVM iOS application, that share a single Firebase backend (Authentication, Realtime Database, Storage) and a single Google Gemini 1.5 Flash model for AI-ranked event feeds and AI-generated trip names. We describe the motivation for the system, the rationale for choosing native-per-platform development over a single cross-platform codebase, the concrete engineering decisions that shape each client (offline persistence, layered .edu email gating, ListAdapter/DiffUtil list rendering on Android, and MVVM with Combine and Swift Concurrency on iOS), and the honestly documented limitations of the system, including a client-embedded Gemini API key and a still-incomplete iOS Firebase integration. NEUQuest was built by a five-person team for the NUMAD24Su Mobile Application Development course at Northeastern University in Summer 2024.

Index Terms—mobile application development, cross-platform architecture, native Android, SwiftUI, MVVM, Firebase, large language models, trip planning, event discovery

I. INTRODUCTION

University students routinely juggle multiple disconnected platforms to find events and activities that fit both their interests and their limited budgets. At Northeastern University, this fragmentation spans paid-event marketplaces such as Eventbrite, informal club Discord servers for free events, the official university events portal, and ad hoc Facebook groups for last-minute meetups. No single surface is tuned specifically to a student’s declared interests and financial constraints, and none combines event discovery with trip-level budget planning.

NEUQuest addresses this gap with a mobile application built exclusively for the Northeastern community, gated by @northeastern.edu and @husky.neu.edu email addresses. The application offers an AI-ranked “Right Now” event feed, category-based browsing, event registration and reporting, a community comment system, and a budget-aware trip planner that auto-matches events to a date range and a spending ceiling and uses a generative model to produce evocative trip names. The system was designed and implemented by a five-person student team (Agllai Papaj, Harshitha Chava, Kaustubha Eluri, Sampada Kulkarni, and

Winston Heinrichs) as the capstone project for NUMAD24Su, Northeastern’s Summer 2024 offering of Mobile Application Development.

This paper documents the resulting system: its motivation, its architecture as two native clients sharing one backend, the specific engineering decisions made along the way, and the limitations the team documented rather than concealed. All facts, figures, and claims in this paper are drawn directly from the project’s README files and docs/ directory (architecture, decisions, limitations, and roadmap records); no user counts, benchmark scores, or performance numbers beyond what is explicitly documented in those files are reported.

II. RELATED WORK

Two broad bodies of practice inform NEUQuest’s design.

A. Cross-Platform vs. Native Mobile Development

Mobile teams generally choose between cross-platform frameworks (e.g., React Native, Flutter) that share a single codebase across Android and iOS, and native development on each platform using each vendor’s preferred toolkit (Java/Kotlin with Android SDK and Material Design; Swift/SwiftUI with Apple’s Human Interface Guidelines). The cross-platform approach reduces duplicated implementation effort at the cost of idiomatic fidelity to each platform’s conventions, while native development yields UI and interaction patterns consistent with platform expectations at the cost of implementing (and maintaining) each feature twice. Architectural patterns such as Model-View-ViewModel (MVVM) and unidirectional, reactive state management (e.g., Apple’s Combine framework, or diff-based list rendering on Android) are common techniques for keeping native codebases maintainable regardless of which side of this tradeoff a team lands on.

B. LLM-Assisted Travel and Itinerary Planning

Large language models are increasingly used as a lightweight ranking or generation layer bolted onto otherwise conventional applications: re-ranking a candidate list of items according to a user’s stated or inferred preferences, or generating short, human-readable text artifacts (names, summaries, descriptions) from structured inputs such as a location, a date range, and a budget. A common and pragmatic pattern – followed by NEUQuest – is to scope the model’s involvement narrowly to non-critical-path features (ranking and naming) so

that the application degrades gracefully (e.g., falling back to a chronological feed or a user-typed name) if the model or its API is unavailable, rather than placing the model in the critical path of authentication, payments, or core navigation.

III. SYSTEM ARCHITECTURE

NEUQuest is a polyglot, two-client system: a native Android application written in Java and a native iOS application written in Swift with SwiftUI, both communicating with a single shared Firebase project and a single shared Gemini 1.5 Flash model. There is no server-side application layer between the clients and Firebase/Gemini beyond Firebase’s own managed services.

A. Why Native-Over-Cross-Platform

The choice of two independent native clients over a single cross-platform codebase (e.g., React Native or Flutter) was driven primarily by the constraints of the NUMAD24Su course, which explicitly required platform-native development (documented as ADR-001 in docs/decisions.md). Beyond satisfying the course requirement, the team recorded a substantive tradeoff: native development yields idiomatic UX on each platform – Material 3 conventions on Android, SF Rounded typography and iOS interaction conventions on iPhone – at the direct cost of implementing and maintaining every feature twice. This cost is mitigated by two invariants that both clients are required to obey: (1) Firebase is the single source of truth, with no domain state persisted client-side beyond display caches, so every read and write goes through Realtime Database, Auth, or Storage; and (2) both clients follow the same layered structure – Models feed Services, which feed ViewModels (or, on Android, Activities coupled to Repositories), which feed Views – so that a shared mental model transfers between the two codebases even though the code itself is not shared. A single Firebase project (ADR-002) means the same events, trips, users, and comments data trees are read and written verbatim by both clients, so a trip created on Android is immediately visible on iOS, at the cost of requiring schema changes to be coordinated across both clients simultaneously (there is no version-skew protocol between them).

B. Shared Data Model

Both clients read and write the following Realtime Database schema directly:

```
/users/{uid}
  name, campus, profileImage, isAdmin
  interests: [string]
  plannedTrips: [tripID]
  eventsAttended: [eventID]

/events/{eventID}
  title, description, category, location, price
  startDate, endDate, startTime, endTime
  image, registerLink, createdBy, isReported
  comments/{commentID}: text, commenterName, timestamp

/trips/{tripID}
  title, location, minBudget, maxBudget
  startDate, endDate, startTime, endTime
  mealsIncluded, transportIncluded
  eventIDs: [eventID]
```

C. Gemini Integration

Gemini 1.5 Flash is used for exactly two purposes on both platforms: (1) event ranking, where both clients submit a user’s declared interests and attended-event history and receive a re-ranked ordering of a candidate set of event IDs for the “Right Now” feed; and (2) trip name generation, where both clients submit a trip’s location, budget range, and dates and receive a three-to-five word evocative trip name. There is no backend proxy in front of Gemini; each client holds its own API key locally (GeminiClient.java on Android, Services/GeminiService.swift on iOS), both excluded from version control. Gemini calls are deliberately kept off every critical path – authentication, payments, and sign-up never depend on a Gemini response.

D. Android Client

The Android client is implemented in Java 8, targets a minimum SDK of API 27 (Android 8.1) and a target SDK of API 34 (Android 14), and uses Material Design 3 with XML layouts and ViewBinding. Its structure follows an MVC pattern with MVVM-adjacent infrastructure: Activities act as Views (there are no Fragments), Repository classes under firebase/repository/ act as Services, and plain Java objects under model/ (Event, Trip, User, Comment) act as Models. Background work runs on a ThreadPoolExecutor combined with Firebase Tasks rather than the deprecated AsyncTask. Image loading uses Glide for hero images and Picasso for thumbnails, an inconsistency the team documented rather than silently accepted. The Android client is documented as feature-complete (“Complete” in the project status table). Figures 1 and 2 show two representative Android client screens: the trip-budget entry form, which writes directly into the /trips/{tripID} schema described above, and the user profile screen, which surfaces the /users/{uid} fields alongside a list of planned trips.

E. iOS Client

The iOS client is implemented in Swift 5.9 targeting a minimum of iOS 17, using SwiftUI under a strict MVVM architecture in which Views contain zero business logic: every screen reads exclusively from @Published state exposed by an @MainActor ObservableObject View-Model, and every side effect (Firebase or Gemini access) is isolated inside a Service singleton. State propagation uses Combine, and asynchronous work uses Swift Concurrency (async/await). Local persistence, where used, is via UserDefaults with JSONEncoder/JSONDecoder. The iOS client is documented as “in progress”: several screens still read from MockData rather than live Firebase calls, with full wire-up tracked as Phase 2 of the project roadmap.

IV. KEY ENGINEERING DECISIONS

The project’s docs/decisions.md records ten dated Architecture Decision Records (ADRs); the ones most salient to the system’s engineering are summarized below.

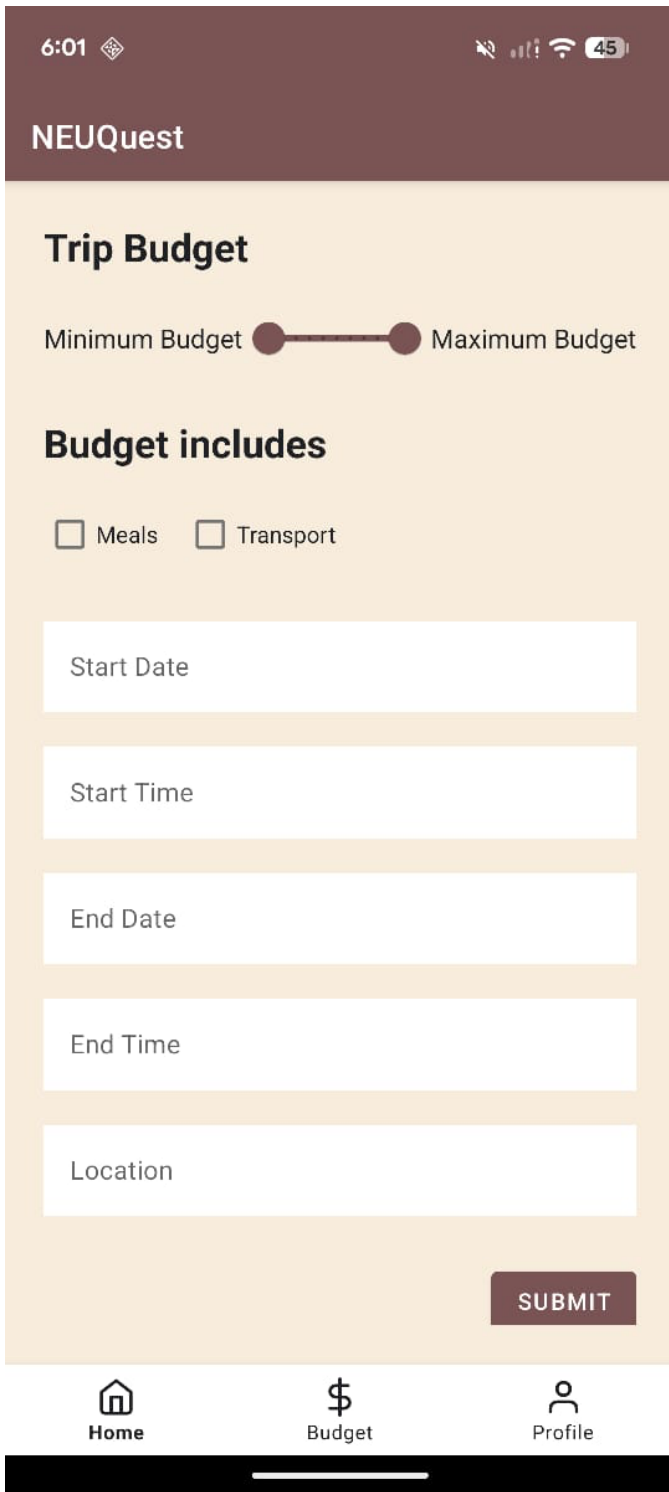


Fig. 1. Android client: the Trip Budget screen, used to create a new entry in the shared `/trips/{tripID}` data tree (budget range, dates/times, location, and meals/transport inclusion flags).

A. Offline Persistence

Android enables `FirebaseDatabase.setPersistenceEnabled(true)` in `NEUQuestApplication` (ADR-006), so events and trips are cached to local disk. This lets a cold app launch render

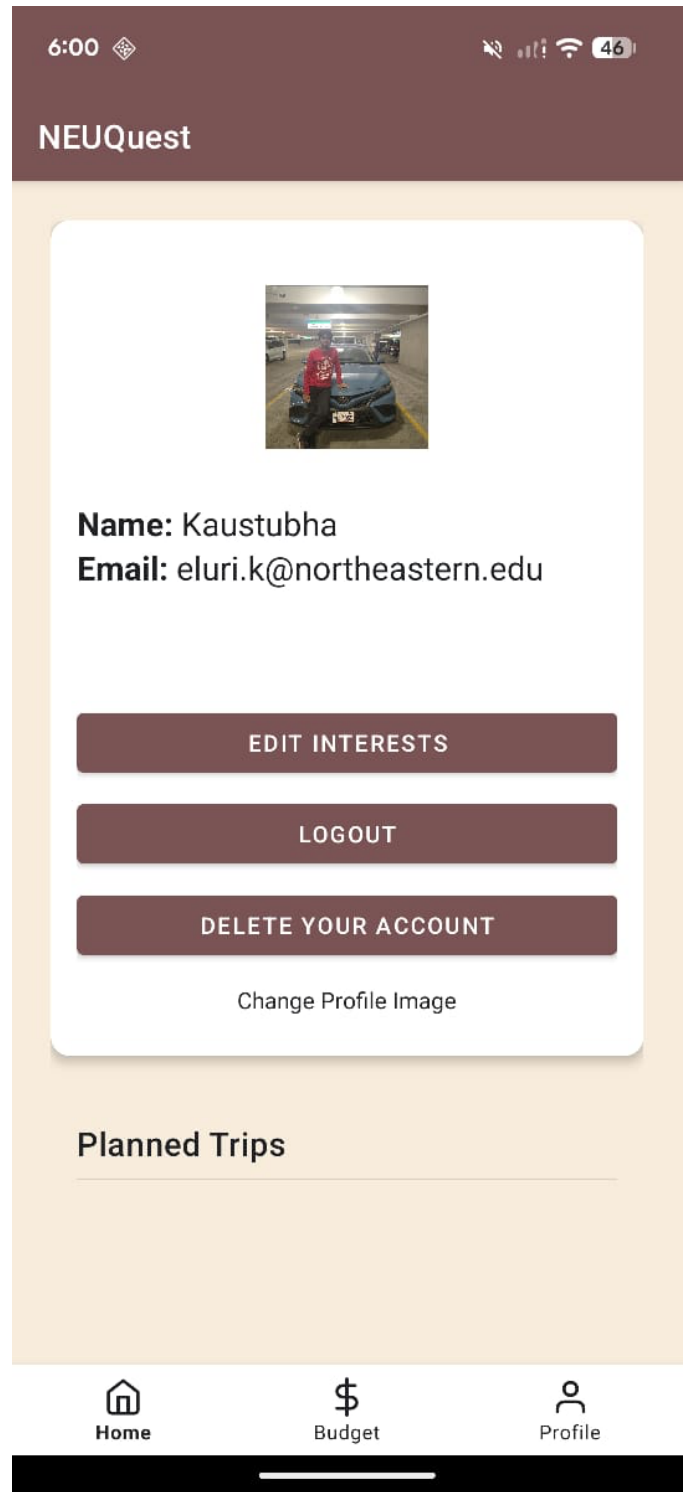


Fig. 2. Android client: the user Profile screen, displaying account fields from `/users/{uid}` (name, email, profile image) along with account management actions and the user's planned trips.

the previous session's data instantly while a background sync with the server proceeds, avoiding a blank loading state on every start; the documented cost is disk usage, described as a typical cache of roughly 5 MB, judged acceptable on modern

Android devices.

B. Layered Auth and .edu Domain Gating

Signup is restricted to @northeastern.edu and @husky.neu.edu addresses through a two-layer defense (ADR-003). The first layer is a client-side check – `SignUpActivity.isValidNeuEmail()` on Android and an equivalent regex-based check in `AuthViewModel.signup(email:password:)` on iOS – that rejects non-NEU addresses before a Firebase Authentication call is ever made. Because Firebase Authentication does not itself expose a server-side rule for domain-restricted signup without a Cloud Function, the team added a second, server-side layer: Realtime Database security rules that reject any write where `auth.token.email` does not end in the required domain suffix. The team explicitly acknowledges that a determined attacker could still bypass the client-side check by calling the Firebase REST API directly, and treats the database rule as the actual backstop rather than the client check.

C. ListAdapter and DiffUtil on Android

All five `RecyclerView` adapters on Android (`EventAdapter`, `TripAdapter`, `CommentsAdapter`, `TimelineEventAdapter`, `AdminConsoleAdapter`) extend `ListAdapter<T, VH>` with a typed `DiffUtil.ItemCallback`, and every list update is driven through `submitList()` rather than the coarse-grained `notifyDataSetChanged()` (ADR-005). This computes a minimal item-level diff and animates only the cells that actually changed, at the documented cost of two additional methods (`areItemsTheSame`, `areContentsTheSame`) per adapter, which the team judged an acceptable tradeoff.

D. MVVM, Combine, and Swift Concurrency on iOS

The iOS client enforces a strict Models → Services → ViewModels → Views layering, with `@MainActor` isolation on both Service singletons and ViewModel ObservableObjects, `@Published` properties propagated via Combine, and asynchronous Firebase and Gemini calls made with Swift’s `async/await`. This mirrors, in spirit, the Android separation of Activities-as-Views and Repositories-as-Services, giving both platforms a comparable logical architecture even though no code is shared between them.

E. AppConstants as a Non-Instantiable Final Class

On Android, all shared constants (SharedPreferences keys, budget-slider bounds, debounce intervals, `RecyclerView` cache size) are centralized in `AppConstants`, implemented as a final class with a private constructor rather than an interface (ADR-007), following the Effective Java pattern for explicit non-instantiable constant holders, avoiding the risk of an interface’s constants being inadvertently exposed through implementation by an unrelated subclass.

F. Gemini API Key in the Client Binary

The Gemini API key is embedded directly in each client’s source (`GeminiClient.java` on Android, `Services/GeminiService.swift` on iOS), both excluded from version control but still present in the compiled binary shipped to a device (ADR-008). The team explicitly labels this decision “demo-grade” and acknowledges that anyone who decompiles either binary can extract the key and consume the team’s Gemini quota; the documented mitigation is a per-key spend cap configured in Google AI Studio, with key rotation as a fallback if abuse occurs. The documented production fix is to route all Gemini calls through a Firebase Cloud Function that validates each client’s Firebase ID token before proxying to Gemini.

V. EVALUATION

NEUQuest was evaluated primarily through its Android test suite and through the team’s own documented quality gates rather than through externally measured user or performance metrics. The Android client includes 30 JUnit/Espresso unit tests covering: eight boundary and cross-year cases for `Event.isWithinDateRange`, four cases for `Event.compareTo` sort ordering, four regex cases for `RightNowActivity.extractTitles`, five cases (including spoofed domains) for `SignUpActivity.isValidNeuEmail`, two null-safety cases for `Trip.addEventID`, and four sanity checks on `AppConstants` values. Documented quality gates additionally require that `./gradlew assembleDebug` passes, that every `RecyclerView` adapter uses `ListAdapter/DiffUtil` rather than `notifyDataSetChanged()`, that Firebase persistence is enabled in `NEUQuestApplication`, that the iOS project builds against the iOS 17 simulator SDK with `xcodebuild`, that NEU email gating is enforced at signup on both clients, and that Gemini calls remain bounded to event ranking and trip naming and are never placed on a critical path. The Android codebase comprises 36 Java source files against six third-party dependencies; the iOS codebase comprises 35+ Swift files with zero third-party dependencies at MVP (Firebase was subsequently added via Swift Package Manager). The project’s own dataset scale is documented as approximately 50 events at the time of writing. No user-count, adoption, or performance-benchmark figures beyond these are documented in the project’s materials, and none are claimed here.

VI. LIMITATIONS

The project’s `docs/limitations.md` enumerates limitations across several categories, of which the following are the most significant.

A. Security

The Gemini API key embedded in the client binary can be extracted by decompiling either app; the only current mitigation is a per-key spend cap in Google AI Studio. The NEU

domain check is enforced client-side with a Realtime Database security-rule backstop, but the team notes that a determined attacker hitting the Firebase REST API directly could still write data if the security rules were ever misconfigured. The application offers no two-factor authentication or passkey support, relies on Firebase Auth’s default six-character password minimum with no additional strength enforcement, and applies no rate limiting to comment posting, event reporting, or trip creation.

B. Cross-Platform Inconsistency

The two clients render different color palettes today: Android uses a dusty-rose primary (approximately #7A5C58) on a cream background after drifting during its Material 3 migration, while iOS retains the original NEU Red (#CC0000) specification (ADR-010). The iOS client still contains mock-data-backed screens rather than full Firebase wiring, while the Android client is fully wired to Firebase. Image loading is also inconsistent: Android uses both Glide and Picasso, while iOS uses a bespoke `AsyncCachedImage` wrapper around `NSCache`.

C. Data, Backend, and Platform Gaps

The event feed loads all events at once with no pagination, which the team notes is viable at the current scale (approximately 50 events) but will not scale further; search is client-side substring matching with no indexing. The README’s claim that “like/dislike signals fine-tune recommendations over time” is explicitly flagged in `limitations.md` as not yet implemented – those signals are not currently captured. The system uses Firebase Realtime Database rather than Firestore, yielding a denormalized schema that makes complex queries awkward, and has no backup or export mechanism, meaning deletion of the Firebase project would be permanent data loss. Neither client implements push notifications, deep linking, Google Maps integration, calendar export, or biometric app-lock. On Android specifically, there is no Jetpack Compose, no AAC `ViewModel` layer, no Navigation Component (navigation is `Intent`-driven and the back stack is described as brittle in places), and no dependency-injection framework. On iOS, some screens remain mock-backed, no app icon set is finalized, and there is no Keychain-backed token storage.

D. Scope

NEUQuest is explicitly a university course project (NUMAD24Su, Group 9, Summer 2024), not deployed to the Play Store or App Store and not intended for production use. It was built by five contributors in a six-week course, and the team notes that code style varies across files as a result, with the demo loop prioritized over edge cases such as locale handling, right-to-left layout, and network-loss recovery.

VII. CONCLUSION AND FUTURE WORK

NEUQuest demonstrates that a fragmented, student-specific problem – discovering affordable, interest-aligned events and planning budget-constrained trips – can be addressed with

two independently implemented native mobile clients unified not by shared code but by a shared backend contract: one Firebase project and one Gemini model, accessed through a consistent Models-Services-ViewModels-Views layering on both platforms. The project’s own documentation treats architectural debt as a first-class artifact: color-palette drift, an unproxied API key, and an incompletely wired iOS client are all recorded as accepted, dated decisions rather than hidden gaps.

The project’s own roadmap lays out an explicit path from course project to shippable product: migrating the Android client from Java/XML to Kotlin and Jetpack Compose with AAC `ViewModel` and `StateFlow` (Phase 1); completing the iOS Firebase wire-up and removing remaining `MockData` paths, including Keychain-backed token storage (Phase 2); introducing a Firebase Cloud Function proxy in front of Gemini and rotating the currently client-embedded API key (Phase 3); reconciling the Android and iOS color palettes into one canonical design system (Phase 4); capturing real like/dislike feedback signals and feeding them into the Gemini ranking prompt (Phase 5); adding push notifications, Google Maps integration, calendar export, and a budget visualization (Phase 6); pursuing Play Store and App Store distribution with production analytics and crash reporting (Phase 7); and, longer-term, a Next.js web client sharing the same Firebase project and Gemini contract (Phase 8). The project’s documentation explicitly places direct ticketing and payments, and competing with general-purpose event platforms such as Eventbrite, out of scope, reaffirming NEUQuest’s identity as a discovery-and-budgeting tool for the Northeastern community rather than a booking platform.

ACKNOWLEDGMENT

The author thanks fellow NUMAD24Su Group 9 team members Agllai Papaj, Harshitha Chava, Sampada Kulkarni, and Winston Heinrichs for their contributions to the NEUQuest project at Northeastern University.